Data Analyst Boot Camp + MySQL Complete Notes

(For interview revision — simple English, easy to remember)

PART 1: How to Become a Data Analyst (Roadmap)

5 Core Skills to Learn (in order of priority)

1. **SQL** – Used to query and retrieve data from a database. Easy to learn, and most companies test SQL in technical interviews.
2. **BI Tool (Tableau / Power BI)** – Used to visualize data and build dashboards. Skills are transferable between different BI tools.
3. **Excel** – Used for cleaning data and making charts. Every company expects you to know basic Excel.
4. **Python** – Used for data manipulation, visualization, web scraping, automation. Harder to learn than SQL/BI but very powerful.
5. **Cloud Platform (AWS / Azure / GCP)** – Becoming more important as companies move data to the cloud.

Steps After Learning Skills

1. **Build Projects** – Take a skill (e.g., Tableau) and build something real with it (e.g., a dashboard). Helps you answer interview questions like "How have you used SQL?" with a real example instead of just "I took a course."
2. **Build a Portfolio Website** – A website to showcase your projects to recruiters/hiring managers. Not mandatory, but helps you stand out and land interviews.
3. **Build a Resume** – Put **Skills** and **Projects** near the top (right after contact info), since you may not have relevant work experience or degree. Work experience/education go lower if not relevant.
4. **Apply for Jobs** – Don't only apply blindly on Glassdoor/Naukri/LinkedIn. Also try to **work with a recruiter** — recruiters help you find jobs and get paid by the company (a % of your salary), so it costs you nothing.
5. **Accept the Job Offer.**

Time Estimates

- Learning all core skills: ~3–4 months (less if skipping Python/cloud)
- Building 3–5 projects: ~3–6 weeks (1–2 weeks per project)

- Building a resume: under 1 week
- Applying + landing a job: ~2–4 months on average
- **Total average: ~6 months** (can be faster with focus)

PART 2: Data Fundamentals

What is Data?

- Data = raw facts and figures (can be a number, word, date — doesn't need to be on a computer).
- Without context, data is mostly useless. Real world example: weather apps and banking apps use lots of collected data points (temperature, location, transactions, etc.) to give useful output.

Types of Data

Type	Description	Example
Structured	Organized in rows & columns	Excel file, SQL table
Semi-structured	Some structure but more complex (nested)	JSON, XML
Unstructured	No fixed format	Images, video, audio

- Structured data = only ~20% of enterprise data. Unstructured = ~80%. A big part of data work is converting unstructured → structured.
- Structured = quantitative (measurable numbers). Unstructured = often qualitative (free text, opinions).

KPI vs Metric

- **Metric** = any measurement that gives information from data (e.g., website visits, units sold).
- **KPI (Key Performance Indicator)** = a *specific* metric that directly tracks progress toward a goal.
- **Rule:** All KPIs are metrics, but not all metrics are KPIs.
- How to choose a good KPI: (1) Know your goal, (2) pick the metric that best tracks progress to that goal, (3) make sure it's actionable (you can actually influence it).

Data Types (Programming/Database Level)

- **String** – free text (names, addresses, categories)
- **Numerical** – Integer (whole numbers) and Decimal (e.g., 12.5)
- **Date/Time** – Date, Time, and Timestamp (date + time combined)
- Knowing data types matters because it decides what operations you can do (e.g., averages on numbers, grouping/counting on strings).

File Types

Category	Examples	Notes
Plain text	TXT, CSV	CSV = Comma Separated Values, simple and very common
Structured	XLSX (Excel), DB (database file)	Rows & columns or schemas
Semi-structured	JSON, XML	Nested data (data within data)
Unstructured	MP4, PNG, JPG	Multimedia, raw
Big Data/Specialized	Parquet	Used in Spark/Databricks for handling huge data efficiently

Data Collection

- **Definition:** Gathering data from various sources for analysis & decision-making.
- Why important: gives raw material for decisions, helps spot trends, and lays foundation for data quality.
- **Data Pipeline / ETL** = Extract, Transform, Load.
    1. **Extract** – pull raw data from sources into a staging area.
    2. **Transform** – clean/reshape the data to make it usable.
    3. **Load** – put it into a database/data warehouse for analysis.
- Data collection is an ongoing process — pipelines break or need updates when source data changes.

Data Cleaning

- **Definition:** Identifying and fixing "dirty data" so it's accurate, consistent, and

complete.

- Why clean data: (1) Accuracy for stakeholders, (2) Efficiency (clean data is easier to use), (3) Builds trust with stakeholders/clients.
- **Common dirty data issues:** inconsistent naming/spelling, missing values, mixed letters & numbers, inconsistent formatting, non-printable characters, incomplete records.
- **Data Cleaning Cycle (steps):**
  1. Import data
  2. Merge data sets
  3. Rebuild missing data
  4. Standardize & normalize
  5. De-duplicate
  6. Verify & enrich (quality check)
  7. Save in final usable format
  - Note: This cycle is not strict/fixed — order can change depending on data; cleaning is an ongoing, repeated process, not "one and done."

PART 3: MySQL — Beginner Series

Setup

- Download MySQL from `dev.mysql.com/downloads/installer`. Install MySQL Server + Workbench (the GUI tool used to write/run SQL).
- A **database** holds **tables**; tables have **rows** (records) and **columns** (fields).
- Main practice database used throughout: **parks_and_recreation** (tables: `employee_demographics`, `employee_salary`, `parks_departments`).

SELECT Statement

- Used to choose which **columns** to display.
- `SELECT * FROM table;` → `*` means "everything" (all columns).
- `SELECT first_name, last_name FROM table;` → select specific columns (comma separated).
- `LIMIT 1000` is added by default by Workbench to avoid loading too many rows at once (keeps performance good).
- Need to specify `database.table` if multiple databases exist and there's ambiguity about which database is "active" (bold one in sidebar).

- Can do calculations in SELECT (e.g., `age + 10`).
- **Order of operations (PEMDAS)**: Parentheses → Exponent → Multiplication → Division → Addition → Subtraction.
- `#` creates a comment (not executed).
- **DISTINCT** – returns only unique values in a column (or unique combination if multiple columns selected).

WHERE Clause

- Filters **rows** (not columns) based on a condition.
- **Comparison operators:** `=`, `>`, `<`, `>=`, `<=`, `!=` (not equal).
- **Logical operators:** `AND` (both conditions true), `OR` (either condition true), `NOT`.
- Parentheses `()` can group conditions to control logic order (similar to PEMDAS for logic).
- **LIKE** – pattern matching, not exact match. Uses wildcards:
  - `%` = anything (any number of characters)
  - `_` = exactly one character
  - Example: `LIKE 'jer%'` = starts with "jer"; `LIKE '%jer%'` = contains "jer" anywhere.

GROUP BY and ORDER BY

- **GROUP BY** – groups rows with same value in a column together so aggregate functions can be applied.
  - If you SELECT a non-aggregated column, it must also appear in GROUP BY, or you'll get an error.
  - Can group by multiple columns.
- **Aggregate functions:** `AVG()`, `MAX()`, `MIN()`, `COUNT()`, `SUM()`.
- **ORDER BY** – sorts the result.
  - Default = `ASC` (ascending, smallest→largest / A→Z).
  - `DESC` = descending (largest→smallest / Z→A).
  - Can order by multiple columns (order of columns matters — first column sorts first).
  - Can use column **position numbers** instead of names (not recommended — risky if columns change later).

HAVING vs WHERE

- **WHERE** filters rows **before** grouping (cannot filter on aggregate functions).
- **HAVING** filters **after** GROUP BY — used specifically to filter on aggregated values (e.g., `HAVING AVG(age) > 40`).
- Both can be used together in one query: WHERE filters raw rows, HAVING filters the grouped/aggregated result.

LIMIT and Aliasing

- **LIMIT n** – restricts output to top n rows. Often combined with ORDER BY for "top N" queries.
- `LIMIT x, y` – skip x rows, then return next y rows (pagination-style).
- **Aliasing (`AS`)** – renames a column (or table) in the output. Useful for readability and to use the new name later in HAVING. The `AS` keyword is optional (implied).

PART 4: MySQL — Intermediate Series

Joins

- **Joins** combine 2+ tables that share a common column (names don't have to match exactly, just compatible data).
- **INNER JOIN** (default `JOIN`) – returns only rows that match in **both** tables.
- **LEFT JOIN (LEFT OUTER JOIN)** – returns **everything from the left table**, plus matches from right table (unmatched = NULL).
- **RIGHT JOIN (RIGHT OUTER JOIN)** – returns **everything from the right table**, plus matches from left table (unmatched = NULL).
- **SELF JOIN** – joining a table to itself (using two aliases, e.g., emp1/emp2). Useful for comparing rows within the same table (e.g., assigning "Secret Santa" by matching employee IDs).
- **Ambiguous column error** – happens when the same column name exists in both joined tables; must specify `table.column` or use aliases.
- **Joining multiple tables** – can chain joins (Table A → Table B → Table C) as long as each pair has a common column.
- **Aliasing tables** (e.g., `dem`, `sal`) makes long queries much easier to read.

UNION

- Combines **rows** from two or more SELECT statements (stacks them vertically, unlike JOIN which goes sideways/columns).
- Column **count and data type** should match/be consistent between the SELECT statements.
- `UNION` (default) = removes duplicate rows (like DISTINCT).
- `UNION ALL` = keeps all rows, including duplicates.
- Real use case shown: combining multiple SELECTs with different filter conditions and labels (e.g., labeling employees as "old man", "old lady", "highly paid employee").

String Functions

Function	What it does
`LENGTH()`	Returns length of a string
`UPPER()` / `LOWER()`	Converts text to all uppercase / lowercase
`TRIM()`	Removes white space from both ends
`LTRIM()` / `RTRIM()`	Removes white space from left only / right only
`LEFT(str, n)`	Takes first n characters from the left
`RIGHT(str, n)`	Takes last n characters from the right
`SUBSTRING(str, start, length)`	Extracts part of string starting at a position for a given length (very powerful — e.g., extracting month from a date string)
`REPLACE(str, old, new)`	Replaces specific characters/text with new ones
`LOCATE(substr, str)`	Finds the position of a substring inside a string
`CONCAT(col1, col2, ...)`	Joins multiple columns/strings into one (e.g., combining first_name + last_name into full_name)

CASE Statement

- Works like an **IF-ELSE** in SQL, used inside SELECT to add logic.

- Syntax: `CASE WHEN condition THEN result WHEN condition2 THEN result2 ELSE default END`.
- Can use `BETWEEN x AND y` inside conditions.
- Common uses: creating labels/categories (e.g., age brackets: "young", "old", "on death's door" 😄), or doing conditional calculations (e.g., calculating raises/bonuses based on salary brackets or department).
- Should alias the resulting column with `AS` for clarity (e.g., `AS age_bracket`).

Subqueries

- A **query inside another query**. Can be used in:
  1. **WHERE clause** – e.g., filter employees whose ID is in a list returned by another query (`WHERE id IN (SELECT ... )`).
  2. **SELECT clause** – e.g., show each salary next to the overall average salary (subquery returns a single value).
  3. **FROM clause** – treat a subquery's result as a temporary table to query further (must give it an alias, e.g., `AS aggregated_table`).
- Important rule: if subquery in WHERE returns more than 1 column, you'll get an error ("operand should contain 1 column").
- Subqueries are powerful but can get hard to read in complex cases — CTEs (next topic) are often cleaner.

Window Functions

- Similar to GROUP BY, **but** don't collapse rows into one — each row stays separate while still calculating an aggregate "over" a group.
- Syntax: `AGG_FUNC(column) OVER (PARTITION BY column ORDER BY column)`.
- `PARTITION BY` = like GROUP BY but doesn't merge rows.
- **Rolling Total (running total):** Using `SUM() OVER (PARTITION BY ... ORDER BY ...)` — adds up values row by row within a partition.
- **ROW_NUMBER()** – gives each row a unique number (no duplicates) within a partition, based on order.
- **RANK()** – gives same rank to ties, but **skips** the next number (e.g., 1,2,3,3,5).
- **DENSE_RANK()** – gives same rank to ties, but does **NOT** skip the next number (e.g., 1,2,3,3,4).
- Window functions are great for things like "Top N per group" or running/cumulative totals.

PART 5: MySQL — Advanced Series

CTE (Common Table Expression)

- Defines a **named, temporary result set** that you can reference right after defining it (within the same query).
- Syntax: `WITH cte_name AS (subquery) SELECT * FROM cte_name;`
- More **readable** than nested subqueries, especially for complex queries.
- Limitation: can only be used **immediately** after being defined — can't reuse later like a saved table (it's not stored anywhere).
- Can create **multiple CTEs** in one query (comma-separated), and even join one CTE to another.
- Can rename ("alias") all resulting columns at once right after the CTE name, e.g., `WITH cte_name (col1, col2) AS (...)`.

Temporary Tables

- A real table that exists only for the current session (disappears when you disconnect/close that session).
- Two ways to create:
  1. `CREATE TEMPORARY TABLE table_name (col1 datatype, col2 datatype, ...);` then `INSERT INTO` manually.
  2. `CREATE TEMPORARY TABLE table_name AS (SELECT ... );` — directly creates and fills it from a query (most commonly used).
- Useful for: storing intermediate results for complex multi-step queries, manipulating data before inserting into a permanent table, used a lot inside **stored procedures**.
- **CTE vs Temp Table:**
  - CTE = simpler, single-use, one level of transformation, good readability.
  - Temp Table = more advanced, reusable within the session, good for complex multi-step logic (e.g., in stored procedures).

Stored Procedures

- A way to **save SQL code** so you can run ("call") it again anytime without rewriting it.
- Syntax basics:

```
sql
```

```
DELIMITER $$
CREATE PROCEDURE procedure_name()
BEGIN
    -- your SQL code here
END $$
DELIMITER ;
```

- **DELIMITER** – temporarily changes the "end of statement" symbol (from `;` to something like `$$`) so MySQL doesn't get confused when the procedure itself contains multiple semicolon-ended statements. Always changed back to `;` after.
- Call it using: `CALL procedure_name();`
- **Parameters** – allow passing input values into a stored procedure.
  - Syntax: `CREATE PROCEDURE proc_name(p_employee_id INT) BEGIN ... WHERE employee_id = p_employee_id; END`
  - Naming convention: prefix parameter names with `p_` to avoid confusion with actual column names.
- Benefits: reusability, simplifies repetitive code, helps performance/organization in real jobs.

Triggers

- A trigger is code that runs **automatically** when a specific event happens on a table (INSERT, UPDATE, DELETE).
- Syntax basics:

sql

```
DELIMITER $$
CREATE TRIGGER trigger_name
AFTER INSERT ON table_name
FOR EACH ROW
BEGIN
    -- action to perform automatically
END $$
DELIMITER ;
```

- `NEW.column_name` – refers to the new value being inserted/updated (used inside the trigger).
- `OLD.column_name` – refers to the value before update/delete.

- Example use case: when a new row is inserted into `employee_salary`, automatically insert matching `employee_id`, `first_name`, `last_name` into `employee_demographics` too — keeps both tables in sync automatically.
- Limitation: MySQL only supports **row-level** triggers (fires once per row), not batch/table-level triggers (unlike SQL Server).

Events

- Similar to triggers, but instead of firing on a data **event** (insert/update), it fires on a **time schedule** (like a cron job).
- Syntax basics:

```sql
CREATE EVENT event_name
ON SCHEDULE EVERY 1 DAY  -- or 1 MONTH, 30 SECOND, etc.
DO
BEGIN
    -- action to perform on schedule
END;
```

- Use cases: scheduled data imports, scheduled report generation/exports, scheduled cleanup/automation tasks.
- Troubleshooting tips if events don't run:
  - Check `SHOW VARIABLES LIKE 'event_scheduler';` — must be `ON`.
  - Check MySQL Workbench preference: SQL Editor → uncheck "Safe Updates" if UPDATE/DELETE statements are being blocked.

PART 6: Project 1 — Data Cleaning (Real World Example)

Dataset: "World Layoffs" (real layoffs data, ~2,361 records, columns: company, location, industry, total_laid_off, percentage_laid_off, date, stage, country, funds_raised_millions).

Golden Rule: Never clean the RAW table directly

- Always create a **staging table** (copy of raw data) and do all cleaning there. Keeps the original raw data safe in case of mistakes — this mirrors real workplace practice (ETL pipelines depend on raw data staying intact).

1. **Remove Duplicates**
   - Used `ROW_NUMBER() OVER (PARTITION BY all_relevant_columns)` inside a CTE to flag duplicate rows (row_num > 1 = duplicate).
   - Problem: MySQL CTEs are **not directly updatable/deletable** — so the workaround is: create a new staging table that includes this `row_num` column, then `DELETE FROM new_table WHERE row_num > 1`.
2. **Standardize the Data**
   - Used `TRIM()` to remove extra white spaces (e.g., around company names).
   - Fixed inconsistent category labels (e.g., "Crypto", "Cryptocurrency", "Crypto " all standardized to one value "Crypto") using `UPDATE ... SET ... WHERE column LIKE '%crypto%'`.
   - Fixed trailing punctuation (e.g., "United States." → "United States") using `TRIM(TRAILING '.' FROM column)`.
   - Converted a **text-based date column** into a proper **DATE** data type:
     - Used `STR_TO_DATE(date_column, '%m/%d/%Y')` to convert text into a real date format.
     - Then used `ALTER TABLE ... MODIFY COLUMN date DATE;` to officially change the column's data type.
3. **Handle NULL and Blank Values**
   - Distinguished between truly NULL fields and empty string/blank fields — sometimes need to convert blanks to NULL first (`UPDATE table SET col = NULL WHERE col = ''`) before logic to "fill missing values" works correctly.
   - Used a **self-join** to populate missing values: if Company X has a blank industry in one row but a filled industry in another row (same company/location), update the blank one using the filled one.
   - Lesson learned: some NULLs **cannot** be populated (e.g., `total_laid_off`, `funds_raised`) because there's no other data source to infer them from — it's fine and expected to leave those as NULL.
4. **Remove Unnecessary Rows/Columns**
   - Rows with **both** `total_laid_off` AND `percentage_laid_off` as NULL were considered not useful and were deleted (a judgment call — not always 100% certain, but reasonable for analysis purposes).
   - Dropped the temporary `row_num` helper column at the end using `ALTER TABLE ... DROP COLUMN row_num;` since it was no longer needed.

Data cleaning is **not a perfectly linear process** — you often go back and forth, make mistakes, and adjust as you discover new issues. This is normal and expected even for experienced analysts.

PART 7: Project 2 — Exploratory Data Analysis (EDA)

Goal: Explore the cleaned "World Layoffs" data without a fixed agenda — find patterns, trends, and interesting insights.

Key Queries & Insights Performed

- `MAX(total_laid_off)` → found the single largest layoff event (12,000 in one event — Google).
- Companies with `percentage_laid_off = 1` (i.e., 100%) → identified companies that completely shut down.
- `GROUP BY company` + `SUM(total_laid_off)` + `ORDER BY DESC` → ranked companies by total layoffs (Amazon, Google, Meta, Microsoft, etc. were top).
- `MIN(date)` / `MAX(date)` → found the data covered March 2020 to March 2023 (start of COVID to ~3 years later).
- `GROUP BY industry` → Consumer and Retail industries were hit hardest (likely due to COVID lockdowns); Manufacturing/Legal/Aerospace were hit least.
- `GROUP BY country` → United States had by far the most layoffs, followed by India.
- `GROUP BY YEAR(date)` → showed 2022 was the worst year, and the first 3 months of 2023 alone were already nearly as bad as all of 2021 (suggests accelerating trend).
- `GROUP BY stage` (company funding stage) → Post-IPO companies (large, established companies) had the most layoffs.

Advanced Queries

- **Rolling Total by Month** – Used `SUBSTRING(date, 1, 7)` to extract "YYYY-MM", grouped and summed layoffs per month, then wrapped it in a CTE with a window function (`SUM() OVER (ORDER BY month)`) to build a running/cumulative total — showed total layoffs climbing from ~9,000 (March 2020) to 383,000+ by March 2023.
- **Top 5 Companies by Layoffs, Per Year** – Used a **multi-level CTE** approach:
  1. First CTE: group by company + year, sum total laid off.

2. Second CTE: apply `DENSE_RANK() OVER (PARTITION BY year ORDER BY total_laid_off DESC)` on top of the first CTE.
3. Final query: filter `WHERE ranking <= 5` to get top 5 companies per year.

- This is a strong example of **chaining multiple CTEs together** for a real analytical use case.

Key Takeaway

EDA often starts simple (basic aggregates, GROUP BY, MAX/MIN) and builds up to more advanced multi-step queries (rolling totals, ranked subsets per group) using window functions and chained CTEs. Both this and the cleaning project are recommended as strong **portfolio projects**.

Quick Reference: SQL Clause Order (Logical Execution Order)

```
FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
```

(Useful to remember when debugging "why doesn't my filter work" issues — e.g., why WHERE can't filter on aggregates, but HAVING can.)

Quick Reference: Common Interview Questions This Video Covers

- What is the difference between WHERE and HAVING?
- What is the difference between INNER JOIN, LEFT JOIN, and RIGHT JOIN?
- What is a self join and when would you use it?
- What is the difference between UNION and UNION ALL?
- What is a CTE and how is it different from a subquery / temp table?
- What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?
- What is a window function and how is it different from GROUP BY?
- What is the difference between a trigger and an event?
- Explain the ETL process (Extract, Transform, Load).
- What is the difference between structured, semi-structured, and unstructured data?
- What is the difference between a metric and a KPI?
- Walk me through your data cleaning process / how do you handle duplicates and NULLs in SQL?